

TRAINING NOTES

WEEK -1:

Basics of Computer Architecture, Hardware & Software

Computer architecture means the basic structure of a computer i.e. is how it's different parts work together to execute programs and process data. The computer's hardware and software work together to perform the task.

Main Parts of a Computer

A computer mainly consists of:

Hardware: Hardware is the physical part of a computer that we can see and touch.

Examples: CPU, RAM, Hard disk / SSD, Keyboard, Mouse, Monitor, Motherboard Printer, Network card.

1.CPU — Central Processing Unit

The CPU is the main processing unit of the computer.

You can think of it as the part responsible for executing instructions and performing calculations.

For example, your C# code:

```
int a = 10;
```

```
int b = 20;
```

```
int result = a + b;
```

When this program runs, the CPU executes the instructions required to perform the operation.

- **ALU** — Arithmetic Logic Unit
- **Control Unit**
- **Registers**
- **Cache**

2. RAM — Random Access Memory: RAM is temporary working memory used while programs are running.

Important point

RAM is generally volatile memory.

That means when the computer loses power, the data in RAM is lost.

Example: Computer ON -> Program is in RAM -> Power OFF -> RAM contents disappear

3.Storage — HDD / SSD

Storage is used for long-term data storage.

RAM vs Storage

RAM	Storage
Temporary working memory	Long-term storage
Ram is Faster.	Usually slower than RAM
Data is lost when power is removed	Data remains after shutdown
Used while programs run	Stores files/programs

4. Motherboard

The motherboard is the main circuit board that connects the different hardware components.

It allows components such as: CPU, RAM, Storage & Other devices to communicate with each other.

5. Input Devices

Input devices allow us to give information or commands to the computer.

Examples: Keyboard, Mouse, Microphone

6. Output Devices

Output devices show or provide the result produced by the computer.

Examples: Monitor, Printer, Speakers, Projector.

What is Software?

Software is a collection of programs and instructions that tells the hardware what to do.

Unlike hardware, software cannot normally be physically touched.

Examples: Windows, Chrome, Visual Studio etc.

PROGRAMMING FUNDAMENTALS & OOPs:

Programming means giving instructions to a computer to perform a particular task.

A programming language such as C# allows us to write these instructions.

Simple C# example

```
using System;
```

```
class Program
```

```

{
static void Main ()
{
    int a = 10;
    int b = 20
    int result = a + b;
Console.WriteLine(result);
}
}

```

Output

30

Here:

- int a = 10 → stores 10
- int b = 20 → stores 20
- a + b → adds them
- result → stores the answer
- Console.WriteLine() → (displays the answer)

2. Variables: It is a named memory location used to store data.

Syntax:

datatype variable Name = value;

Example:

int age = 21;

string name = "Devika";

double salary = 45000.50;

3. Data Types: A data type tells C# what kind of data a variable can store.

There are two major categories

1. Value Types
2. Reference Types

1. Value Types: A value-type variable directly contains its value.

Common value types:

Type	Used for	Example
int	Whole numbers	25
float	Decimal numbers	10.5f
double	Larger decimal values	25.75
decimal	Financial/high precision	50000.50m
char	Single character	'A'
bool	True/False	true

2. Reference Types: Reference types store a reference to an object in memory.

Common reference types:

- string
- object
- arrays
- classes
- interfaces
- delegates

Example:

```
string name = "Devika";
```

Here name refers to a string object.

5. Constants

A constant is a value that cannot be changed after it is declared.

We use the const keyword.

6. Operators: Operators are symbols used to perform operations on values.

Main types:

└─ Arithmetic

└─ Relational

└─ Logical

- |— Assignment
- |— Increment/Decrement
- └— Conditional

6.1 Arithmetic Operators: Used for mathematical operations. (Addition, subtraction, division, multiplication, division)

Example

using System;

class Program

```
{
    static void Main ()
    {
        int a = 10;
        int b = 3;

        Console.WriteLine(a + b);
        Console.WriteLine(a - b);
        Console.WriteLine(a * b);
        Console.WriteLine(a / b);
        Console.WriteLine(a % b);
    }
}
```

7. Relational Operators: Used to **compare two values**.

Operator	Meaning
==	Equal
!=	Not equal
>	Greater than
<	Less than
>=	Greater than or equal

Operator	Meaning
<=	Less than or equal

Example:

```
using System;

class Program
{
    static void Main ()
    {
        int age = 21;
        Console.WriteLine(age >= 18);
    }
}
```

Output:

True

Because 21 is greater than 18.

8. Logical Operators: Used to combine conditions.

Example:

```
using System;

class Program
{
    static void Main ()
    {
        int age = 25;
        bool hasId = true;
        if (age >= 18 && hasId)
```

```

    {
        Console.WriteLine("Allowed");
    }
    else
    {
        Console.WriteLine("Not Allowed"); } } }

```

Both conditions must be true because we used &&

10. Type Conversion

Sometimes we need to convert one data type into another.

For example:

string → int

int → double

double → int

Common methods:

Convert.ToInt32()

Convert.ToDouble()

Convert.ToString()

Example:

```
int number = Convert.ToInt32("100");
```

Now:

```
number = 100
```

Conditional Statements: Conditional statements allow the program to make decisions.

Main types:

- ├─ if
- ├─ if-else
- ├─ else if
- └─ switch

if- Statement: Used when we want to execute something only when a condition is true.

Complete example

```

using System;
class Program
{
    static void Main ()
    {
        int age = 20;
        if (age >= 18)
        {
            Console.WriteLine("Adult");
        }
    }
}

```

if-else Statement: Used when there are two possible outcomes.

```

        else
        {
            Console.WriteLine("Minor");
        }
    }
}

```

else-if Statement: Used when there are multiple conditions.

Complete example

```

using System;
class Program
{
    static void Main ()
    {
        int marks = 85;
        if (marks >= 90)
        {

```

```

        Console.WriteLine("A+");
    }
    else if (marks >= 75)
    {
        Console.WriteLine("A");
    }
    else if (marks >= 60)
    {
        Console.WriteLine("B");
    }
    else
    {
        Console.WriteLine("C");
    }
}
}

```

Switch Statement: it is useful when we compare one value against multiple fixed choices.

Complete example

```

using System;

class Program
{
    static void Main ()
    {
        int day = 2;
        switch (day)
        {
            case 1:
                Console.WriteLine("Monday");
                break;

```

case 2:

```
Console.WriteLine("Tuesday");
```

```
    break;
```

case 3:

```
    Console.WriteLine("Wednesday");
```

```
    break;
```

default:

```
    Console.WriteLine("Invalid day");
```

```
    break;
```

```
    }
```

```
    }
```

```
}
```

Output:

Tuesday

Loops: A loop repeats a block of code.

├── for

├── while

└── do-while

for Loop: Used when we generally know how many times something should repeat.

Complete example

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main ()
```

```
    {
```

```
        for (int i = 1; i <= 5; i++)
```

```
        {
```

```
            Console.WriteLine(i);} } }
```

while Loop: Used when repetition depends on a condition.

Complete example

```
using System;

class Program {

    static void Main () {

        int i = 1;

        while (i <= 5) {

            Console.WriteLine(i);

            i++;

        }

    } }
```

do-while Loop: A do-while loop executes at least once, because the condition is checked after the body.

Example:

```
using System;

class Program {

    static void Main () {

        int i = 1;

        do {

            Console.WriteLine(i);

            i++;

        }

        while (i <= 5);

    } }
```

Break and Continue

break: Stops the loop completely.

Complete example

```
using System;

class Program {
```

```

static void Main () {
    for (int i = 1; i <= 10; i++) {
        if (i == 5) {
            break;}
        Console.WriteLine(i);
    } }

```

Continue: Skips the current iteration and continues with the next one.

Example

```

using System;

class Program
{
    static void Main ()
    {
        for (int i = 1; i <= 5; i++)
        {
            if (i == 3)
            {
                continue;
            }

            Console. WriteLine(i);} } }

```

Arrays: An array stores **multiple values of the same type**.

Complete example

```

using System;

class Program
{
    static void Main ()
    {
        int [] marks = {80, 90, 75, 85};
    }
}

```

```

        for (int i = 0; i < marks. Length; i++)
        {
            Console. WriteLine(marks[i]);
        }
    }
}

```

Methods: A method is a block of code that performs a particular task.

This makes programs easier to understand and maintain.

Complete example

```

using System;

class Program
{
    static void AddNumbers ()
    {
        int a = 10;
        int b = 20;
        int result = a + b;
        Console. WriteLine(result);
    }

    static void Main ()
    {
        AddNumbers (); } }

```

Method Parameters: A parameter allows us to send data into a method.

Complete example

```

using System;

class Program
{
    static void Greet (string name)
    {

```

```

        Console.WriteLine("Hello " + name);
    }
static void Main ()
{
    Greet("Devika");
    Greet("Ravi");
}
}

int result = Add (10, 20);

Console.WriteLine(result);
}
}

```

What is OOP?

OOP = Object-Oriented Programming.

It is a programming approach where we organize programs around **objects and classes**.

In C#, we represent this using a class and objects.

Class: A class is a blueprint/template for creating objects.

```

class Employee
{
    public string Name;
    public int Age;
}

```

This is a class.

Object: An object is an actual instance of a class.

Complete example

```

using System;

class Employee
{

```

```

    public string Name;

    public int Age;
}

class Program
{
    static void Main ()
    {
        Employee employee1 = new Employee ()
        employee1.Name = "Devika";
        employee1.Age = 21;
        Console.WriteLine(employee1.Name);
        Console.WriteLine(employee1.Age);
    }
}

```

Constructor: A constructor is a special method that runs automatically when an object is created. It has the **same name as the class**.

Complete example

```

using System;

class Employee
{
    public string Name;
    public int Age;

    public Employee ()
    {
        Name = "Unknown";
        Age = 0;
    }
}

```

```

class Program
{
    static void Main ()
    {
        Employee employee = new Employee ();
        Console.WriteLine(employee. Name);
        Console.WriteLine(employee. Age);
    }
}

```

Encapsulation: Encapsulation means keeping data and the methods that operate on that data together, while controlling how the data is accessed.

Complete example

```

using System;

class Employee
{
    private double salary;

    public void Setsalary (double amount)
    {
        if (amount > 0)
        {
            salary = amount;
        }
    }

    public double GetSalary ()
    {
        return salary;
    }
}

class Program
{

```

```

static void Main ()
{
    Employee employee = new Employee ();
    employee. SetSalary (45000);
    Console.WriteLine(employee. GetSalary ());
} }

```

The field: private double salary, cannot be directly accessed from outside the class.

This is encapsulation.

Inheritance: Inheritance allows one class to acquire members of another class.

Complete example

```
using System;
```

```
class Employee
```

```

{
    public string Name = "Devika";
    public void Work ()
    {
        Console.WriteLine("Employee is working");
    }
}

```

```
class Manager: Employee
```

```

{
    public void Manage ()
    {
        Console.WriteLine("Manager is managing the team");
    }
}

```

```
class Program
```

```

{
    static void Main ()

```

```
{
    Manager manager = new Manager ();
    Console.WriteLine(manager. Name);
    manager. Work ();
    manager. Manage ();}}
```

Types of Inheritance

Common forms are:

1. Single
2. Multilevel
3. Hierarchical
4. Multiple
5. Hybrid

C# does not support multiple inheritance through classes. It can achieve similar behaviour using interfaces.

Polymorphism: Polymorphism means one name, multiple forms.

Main types:

```
|—— Compile-time
|  |—— Method Overloading
|—— Runtime
    |—— Method Overriding
```

Method Overloading: Same method name but different parameters.

Complete example

using System;

class Calculator

```
{
    public int Add (int a, int b)
    {
        return a + b;
```

```

    }
    public int Add (int a, int b, int c)
    {
        return a + b + c;
    }
}
class Program
{
    static void Main ()
    {
        Calculator calculator = new Calculator ();
        Console.WriteLine(calculator. Add (10, 20));
        Console.WriteLine(calculator. Add (10, 20, 30));
    }
}

```

Method Overriding: A child class provides its own implementation of a parent method.

We use virtual & override

Complete example

```

using System;
class Employee
{
    public virtual void Work ()
    {
        Console.WriteLine("Employee is working");
    }
}
class Developer: Employee
{
    public override void Work ()

```

```

    {
        Console.WriteLine("Developer is writing code");
    }
}

class Program
{
    static void Main ()
    {
        Developer developer = new Developer ();
        developer. Work ();
    }
}

```

Abstraction: Abstraction means showing only the important information and hiding unnecessary implementation details.

In C#, abstraction can be achieved using:

1. Abstract classes
2. Interfaces

Abstract Class: An abstract class cannot normally be directly instantiated.

It can contain abstract methods that child classes must implement.

Complete example

using System;

abstract class Vehicle

```

{
    public abstract void Start ();
}

```

class Car: Vehicle

```

{
    public override void Start ()
    {

```

```

        Console.WriteLine("Car starts with a key");
    }
}
class Program
{
    static void Main ()
    {
        Car car = new Car ();

        car. Start ();
    }
}

```

Interface: An interface defines a contract that a class agrees to implement.

Complete example

```

using System;
interface IPayment
{
    void Pay ();
}
class CreditCardPayment: IPayment
{
    public void Pay ()
    {
        Console.WriteLine("Payment made using credit card");
    }
}
class Program
{
    static void Main ()

```

```

{
    CreditCardPayment payment = new CreditCardPayment ();
    payment. Pay ();
}
}

```

Access Modifiers: Access modifiers control where members can be accessed.

Example:

```

class Employee
{
    private int salary;
    public string Name;
    protected int employeeId;
}

```

This Keyword: This refers to the current object.

It is commonly used when parameter and field names are the same.

Complete example

```

using System;
class Employee
{
    private string name;

    public Employee (string name)
    {
        this.name = name;
    }

    public void Display ()
    {
        Console.WriteLine(this.name);
    }
}

```

```

    }
}
class Program
{
    static void Main ()
    {
        Employee employee = new Employee("Devika");
        employee. Display ();
    }
}

```

Static: static members belong to the class itself, rather than a particular object.

Complete example

```

using System;
class Calculator
{
    public static int Add (int a, int b)
    {
        return a + b;
    }
}
class Program
{
    static void Main ()
    {
        int result = Calculator.Add(10, 20);

        Console.WriteLine(result);
    }
}

```

Calculator.Add()

instead of:

Calculator calculator = new Calculator ();

Exception Handling: Exceptions are runtime errors that can be handled so the application doesn't unexpectedly terminate.

Keywords:

1. try
2. catch
3. finally
4. throw

Complete example

```
using System;
```

```
class Program
```

```
{
```

```
    static void Main ()
```

```
    {
```

```
        try
```

```
        {
```

```
            int a = 10;
```

```
            int b = 0;
```

```
            int result = a / b;
```

```
            Console.WriteLine(result);
```

```
        }
```

```
        catch (DivideByZeroException)
```

```
        {
```

```
            Console.WriteLine("Cannot divide by zero");
```

```
        }
```

```
        finally
```

```
        {
```

```
            Console.WriteLine("Program completed");
```

```
}  
}
```

Collections: Collections store multiple objects/data.

Important C# collections include:

List

Dictionary

Queue

Stack

HashSet

List

Stores multiple values dynamically.

```
List<string> names = new List<string> ();
```

Complete example

```
using System;
```

```
using System.Collections.Generic;
```

```
class Program
```

```
{
```

```
    static void Main ()
```

```
    {
```

```
        List<string> names = new List<string> ();
```

```
        names.Add("Devika");
```

```
        names.Add("Ravi");
```

```
        names.Add("Anil");
```

```
        foreach (string name in names)
```

```
        {
```

```
            Console.WriteLine(name);
```

```
        }
```

```
    }
```

```
}
```

Properties in C#

What is a property?

A **property** is a way to safely access the data of a class. Instead of allowing other classes to directly change the data, we use a **property**.

Simple example

```
class Employee
{
    public string Name {get; set;}
}
```

Here:

Name

↓

Property

get means read the value.

set means assign/change the value.

Complete Property Example

```
using System;
class Employee
{
    private string name;
    private int age;
    public string Name
    {
        get
        {
            return name;
        }
    }
}
```

```

        set
        {
            name = value;
        }
    }

    public int Age
    {
        get
        {
            return age;
        }
        set
        {
            age = value;
        }
    }
}

class Program
{
    static void Main ()
    {
        Employee employee = new Employee ();
        employee. Name = "Devika";
        employee. Age = 22;

        Console.WriteLine("Employee Name: " + employee. Name);
        Console.WriteLine("Employee Age: " + employee. Age);
    }
}

```

Indexers in C#: An indexer allows an object to be accessed like an array.

Example

```
using System;

class StudentCollection
{
    private string [] students = new string [3];

    public string this [int index]
    {
        get
        {
            return students[index];
        }

        set
        {
            students[index] = value;
        }
    }
}

class Program
{
    static void Main ()
    {
        StudentCollection students = new StudentCollection ();

        students [0] = "Devika";
        students [1] = "Rahul";
        students [2] = "Priya";
        Console.WriteLine(students [0]);
        Console.WriteLine(students [1]);
    }
}
```

```
        Console.WriteLine(students [2]);  
    }
```

Delegates in C#

What is a Delegate?

A delegate is a **type-safe reference to a method**

Delegate Syntax

First create the delegate:

```
delegate void MyDelegate ();
```

Then create a method with matching signature:

```
static void Hello ()  
{  
    Console.WriteLine("Hello");  
}
```

Then assign:

```
MyDelegate d = Hello;
```

Then call:

```
d ();  
SayWelcome ()
```

and:

```
d ()  
calls SayWelcome ().
```

Delegate with Parameters: A delegate can also point to methods having parameters.

```
using System;
```

```
class Program
```

```
{  
    delegate int Calculate (int a, int b);  
    static int Add (int a, int b)  
    {  
        return a + b;  
    }  
}
```

```

    }
static int Multiply (int a, int b)
{
    return a * b;
}
static void Main ()
{
    Calculate operation;
    operation = Add;
    Console.WriteLine("Addition: " + operation (10, 5));
    operation = Multiply
    Console.WriteLine("Multiplication: " + operation (10, 5));
}
}

```

WEEK-2

DESIGN PRINCIPLES

The most important principles you'll hear in C#/.NET are **SOLID principles**.

SOLID PRINCIPLES

SOLID has five principles:

S → Single Responsibility Principle

O → Open/Closed Principle

L → Liskov Substitution Principle

I → Interface Segregation Principle

D → Dependency Inversion Principle

DESIGN PATTERNS

What is a Design Pattern?

A design pattern is a commonly used solution structure for a recurring software design problem.

Types of Design Patterns: Traditionally, design patterns are grouped into:

├— Creational
├— Structural
└— Behavioural

Singleton: Ensures a class has one shared instance.

Example:

using System;

class Logger

```
{  
    private static Logger instance;  
  
    private Logger ()  
    {  
public static Logger GetInstance ()  
    {  
        if (instance == null)  
    {  
        instance = new Logger ();  
        }  
        return instance;  
    }  
    public void Log (string message)  
    {  
        Console.WriteLine(message);  
    }  
}  
}class Program  
{  
    static void Main ()  
    {  
        Logger logger1 = Logger.GetInstance();
```

```
    Logger logger2 = Logger.GetInstance();  
    logger1.Log("Application started.");  
    Console.WriteLine(logger1 == logger2);  
}  
}
```

RDBMS FUNDAMENTALS

RDBMS means: Relational Database Management System. It stores data in tables and allows relationships between tables.

Important RDBMS Terms

Table: Stores data in rows and columns.

Row: One complete record.

Column: One attribute.

Primary Key: Uniquely identifies each row.

Foreign Key: Creates a relationship between tables.

Constraint: Rules applied to data.

Creating Tables and Data Types

Example:

```
CREATE TABLE Employees  
(  
    EmployeeId INT PRIMARY KEY,  
    EmployeeName VARCHAR (100) NOT NULL,  
    Salary DECIMAL (10,2),  
    JoiningDate DATE,  
    IsActive BIT  
);
```

CREATING DATABASES

```
CREATE DATABASE EmployeeDB;
```

Then:

USE EmployeeDB;

Then create table:

CREATE TABLE Employees

(

EmployeeId INT PRIMARY KEY,

EmployeeName VARCHAR (100),

Department VARCHAR (100)

);

ENFORCING DATA INTEGRITY

Data integrity means: Data should remain accurate, valid, and consistent.

Main types:

Entity Integrity: Primary key ensures unique records.

EmployeeId INT PRIMARY KEY

Referential Integrity: Foreign key maintains relationships.

FOREIGN KEY (DepartmentId)

REFERENCES Departments (DepartmentId)

Domain Integrity: Ensures values are valid.

Example:

Salary DECIMAL (10,2)

Constraints

Important constraints:

PRIMARY KEY

FOREIGN KEY

NOT NULL

UNIQUE

CHECK

DEFAULT

Example:

CREATE TABLE Employees

```
(  
    EmployeeId INT PRIMARY KEY,  
    Name VARCHAR (100) NOT NULL,  
    Age INT CHECK (Age >= 18),  
    Department VARCHAR (50) DEFAULT 'IT'  
);
```

PARTITIONS: Partitioning means dividing a large table/index into smaller logical pieces

RANK FUNCTIONS: Ranking functions assign a position to rows.

Important functions:

1. ROW_NUMBER ()
2. RANK ()
3. DENSE_RANK ()
4. NTILE ()

Example:

```
SELECT EmployeeName, Salary, RANK () OVER (ORDER BY Salary DESC) AS Salary  
Rank
```

```
FROM Employees;
```

If salaries are:

100000

90000

90000

80000

RANK () gives:

100000 → 1

90000 → 2

90000 → 2

80000 → 4

Notice ranks 3 is skipped.

RETRIEVING DATA

The most basic query:

SELECT * FROM Employees;

Specific columns:

SELECT EmployeeName, Department
FROM Employees;

With condition:

SELECT *
FROM Employees
WHERE Department = 'IT';

Sorting:

SELECT *
FROM Employees
ORDER BY EmployeeName;

MODIFYING DATA

INSERT

INSERT INTO Employees
(EmployeeId, EmployeeName, Department)
VALUES
(1, 'Devika', 'IT');

UPDATE

UPDATE Employees
SET Department = 'HR'
WHERE EmployeeId = 1;

DELETE

DELETE FROM Employees
WHERE EmployeeId = 1;

Very important:

UPDATE Employees
SET Department = 'HR';

without WHERE can update **every row**.

Similarly:

DELETE FROM Employees

JOINS: Joins combine data from multiple tables.

INNER JOIN: Returns matching records.

SELECT

e. EmployeeName,

d.DepartmentName

FROM Employees e

INNER JOIN Departments d

ON e. DepartmentId = d. DepartmentId;

LEFT JOIN: Returns all records from the left table and matching records from the right table.

SELECT e. EmployeeName, d. DepartmentName

FROM Employees e

LEFT JOIN Departments d

ON e. DepartmentId = d. DepartmentId;

RIGHT JOIN: Returns all records from the right table and matching records from the left table.

SELECT

e. EmployeeName,

d.DepartmentName

FROM Employees e

RIGHT JOIN Departments d

ON e. DepartmentId = d. DepartmentId;

FULL OUTER JOIN: Returns Matching records + Unmatched records from left+
Unmatched records from right

SELECT *

FROM Employees e

FULL OUTER JOIN Departments d

ON e. DepartmentId = d. DepartmentId;

T-SQL PROGRAMMING

T-SQL is SQL Server's programming language based on SQL.

It supports:

- variables
- conditions
- loops
- error handling
- functions
- stored procedures
- transactions

VARIABLE DECLARATION

Syntax:

```
DECLARE @variable_name datatype;
```

Example:

```
DECLARE @EmployeeName VARCHAR (100);
```

```
SET @EmployeeName = 'Devika';
```

```
SELECT @EmployeeName;
```

Output:

Devika

You can also initialize:

```
DECLARE @Age INT = 22;
```

```
SELECT @Age;
```

IF...ELSE

```
DECLARE @Age INT = 22;
```

```
IF @Age >= 18
```

```
BEGIN
```

```
    PRINT 'Adult';
```

```
END
```

```
ELSE
```

```
BEGIN
```

```
    PRINT 'Minor';
```

```
END
```

Output:

Adult

FUNCTIONS: A function is reusable SQL logic that returns a value or table.

Two major categories:

1. Scalar-valued function
2. Table-valued function

Scalar Function

Returns one value.

```
CREATE FUNCTION GetBonus
```

```
(
```

```
    @Salary DECIMAL (10,2)
```

```
)
```

```
RETURNS DECIMAL (10,2)
```

```
AS
```

```
BEGIN
```

```
    RETURN @Salary * 0.10;
```

```
END;
```

Use:

```
SELECT dbo. GetBonus (50000);
```

Result:

5000

STORED PROCEDURES: A stored procedure is a saved set of SQL statements that can be executed when needed.

Example:

```
CREATE PROCEDURE GetEmployees
```

```
AS
```

BEGIN

SELECT *

FROM Employees;

END;

Execute:

EXEC GetEmployees;

With parameter

CREATE PROCEDURE Get Employee

@EmployeeId INT

AS

BEGIN

SELECT *

FROM Employees

WHERE EmployeeId = @EmployeeId;

END;

Execute:

EXEC Get Employee 1;

TRIGGERS: A trigger is automatically executed when a specified database event happens.

Events can include:

INSERT

UPDATE

DELETE

Example:

CREATE TRIGGER trg_EmployeeInsert

ON Employees

AFTER INSERT

AS

BEGIN

PRINT 'Employee inserted';

END;

When someone runs:

```
INSERT INTO Employees  
VALUES (10, 'Devika', 'IT');
```

the trigger executes automatically.

INDEXES: An index helps SQL Server find rows faster.

Common concepts:

1. **Clustered Index**
2. **Non-clustered Index**

A table can have only one clustered index, while it can have multiple nonclustered indexes, subject to SQL Server's limits and design considerations.

Example:

```
CREATE INDEX IX_Employees_Name  
ON Employees (EmployeeName);
```

Now searching by EmployeeName can benefit from the index.

CURSORS: A cursor allows processing query results row by row.

Example:

```
DECLARE EmployeeCursor CURSOR FOR  
SELECT EmployeeName  
FROM Employees;  
OPEN EmployeeCursor;  
DECLARE @Name VARCHAR (100);  
FETCH NEXT FROM EmployeeCursor INTO @Name;  
WHILE @@FETCH_STATUS = 0  
BEGIN  
    PRINT @Name;  
    FETCH NEXT FROM EmployeeCursor INTO @Name;  
END;  
CLOSE EmployeeCursor;  
DEALLOCATE EmployeeCursor;
```

Important

Cursors can be useful in special cases but set-based SQL is generally preferred when the operation can be expressed that way because row-by-row processing can be slower and more resource-intensive.

TRANSACTIONS: A transaction groups multiple database operations into one logical unit of work.

ACID Properties

Transactions follow four important properties:

A → Atomicity

C → Consistency

I → Isolation

D → Durability

Atomicity: All operations happen or none happen.

Consistency: Database remains valid.

Isolation: Concurrent transactions should not incorrectly interfere with each other.

Durability: Committed changes remain stored even after failures, subject to the database's durability mechanisms.

Complete Transaction Example

```
BEGIN TRANSACTION;
```

```
BEGIN TRY
```

```
    UPDATE Accounts
```

```
        SET Balance = Balance - 1000
```

```
        WHERE AccountId = 1;
```

```
    UPDATE Accounts
```

```
        SET Balance = Balance + 1000
```

```
        WHERE AccountId = 2;
```

```
    COMMIT TRANSACTION;
```

```
PRINT 'Transaction completed successfully';
```

```
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    PRINT 'Transaction failed';
END CATCH;
```

INTRODUCTION TO LINQ

What is LINQ?

LINQ means: **Language Integrated Query**

It allows us to query and manipulate data using C# syntax.

Normally, if we have a list of employees:

Devika IT 50000

Rahul HR 40000

Priya IT 60000

and we want only IT employees; we can use LINQ.

```
var result = employees. Where (e => e. Department == "IT");
```

So, LINQ allows us to work with data in a simple way.

Basic LINQ Example

```
using System;
using System.Collections.Generic;
using System.Linq;
class Employee
{
    public string Name {get; set;}
    public string Department {get; set;}
    public int Salary {get; set;}
}
class Program
{
```

```

static void Main ()
{
    List<Employee> employees = new List<Employee>
    {
        new Employee {Name = "Devika", Department = "IT", Salary = 50000},
        new Employee {Name = "Rahul", Department = "HR", Salary = 40000},
        new Employee {Name = "Priya", Department = "IT", Salary = 60000}
    };
    var result = employees
        .Where (e => e. Department == "IT");
    foreach (var employee in result)
    {
        Console.WriteLine(employee. Name);
    }
}

```

LINQ to SQL: it is a technology that allows developers to query SQL Server data using LINQ and C# objects rather than writing every query manually.

RESTFUL WEB SERVICES: Now we're moving from C# and SQL into web/API development. A web service allows applications to communicate over a network.

REST means: Representational State Transfer

REST is an architectural style used to build web APIs.

A REST API commonly uses HTTP methods such as:

- GET → retrieve data
- POST → create data
- PUT → update/replace data
- PATCH → partially update data
- DELETE → delete data

Example REST API

Suppose we have employees.

GET

/Api/employees

means:

Give me employees.

GET

/Api/employees/10

means:

Give me employee 10.

POST

/api/employees

means:

Create an employee.

PUT

/api/employees/10

means:

Update employee 10.

DELETE

/api/employees/10

means:

Delete employee 10.

HTTP Status Codes

Your API can return status codes.

Important ones:

200 → OK

201 → Created

204 → No Content

400 → Bad Request

401 → Unauthorized

403 → Forbidden

404 → Not Found

500 → Internal Server Error

Example: If employee ID 100 doesn't exist GET /api/employees/100

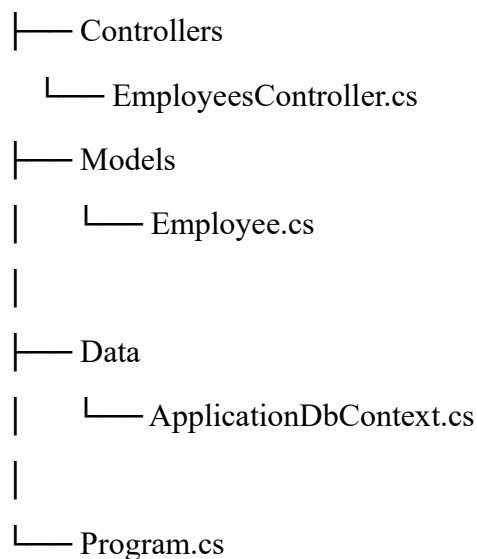
404 Not Found

IMPLEMENTING WEB APIs IN ASP.NET CORE: An ASP.NET Core Web API project normally contains things like: Controllers, Models Services, Data.

Program.cs

A simplified structure:

EmployeeAPI



Simple ASP.NET CORE WEB API: A controller can look like this:

using Microsoft.AspNetCore.Mvc;

namespace EmployeeAPI.Controllers

```
{
    [ApiController]
    [Route("api/[controller]")]
    public class EmployeesController: ControllerBase
    {
        [HttpGet]
        public IActionResult GetEmployees ()
        {
```

```

        var employees = new []
        {
            new {Id = 1, Name = "Devika"},
            new {Id = 2, Name = "Rahul"}
        }
    return Ok(employees);
}
}
}

```

When you call:

GET /api/employees

the API can return JSON such as:

```

[
  {
    "id": 1,
    "name": "Devika"
  },
  {
    "id": 2,
    "name": "Rahul"
  }
]

```

[ApiController]: This attribute tells ASP.NET Core that this controller is intended for API behaviour. It also provides useful features such as automatic API-focused behaviour and model-validation handling. [ApiController]

CONTROLLERS: A controller receives requests and decides what response should be returned.

Example:

```

public class EmployeesController: ControllerBase
{
}

```

For MVC applications with views, controllers commonly inherit from:

Controller

For APIs:

ControllerBase

ACTIONS / ACTION METHODS: An action method is a method inside a controller that handles an HTTP request.

Example:

[HttpGet]

```
public IActionResult GetEmployees()
```

```
{
```

```
    return Ok ();
```

```
}
```

Here:

[HttpGet]

↓

HTTP GET request

GetEmployees ()

↓

Action method

Another example:

[HttpDelete("{id}")]

```
public IActionResult DeleteEmployee (int id)
```

```
{
```

```
    return Ok ();
```

```
}
```

VIEWS: Views are mainly associated with ASP.NET Core MVC.

A View is the UI that is presented to the user.

ASP.NET Core commonly uses Razor views.

File:

Index. Cshtml

Example:

```
<h1>Employee List</h1>
```

```
<p>Welcome to Employee Management</p>
```

The. cshtml file can contain:

HTML Razor syntax

MVC: MVC means:

M → Model

V → View

C → Controller

Model

Represents application data.

Example:

```
public class Employee
{
    public int Id {get; set;}
    public string Name {get; set;}
}
```

View: Displays the UI.

Controller: Handles requests and coordinates the responses.

What is Model Binding?

Model binding automatically takes data from an HTTP request and puts it into C# parameters or objects.

For example, request:

```
/api/employees/10
```

Controller:

```
[HttpGet("{id}")]
```

```
public IActionResult GetEmployee (int id)
```

```
{
```

```
    return Ok(id);  
}
```

ASP.NET Core automatically takes:

10

from the URL and puts it into:

id

DATA ANNOTATIONS: Data annotations are attributes placed on model properties to describe validation rules and metadata.

Common attributes:

[Required]

[StringLength]

[Range]

[EmailAddress]

[Phone]

[Compare]

Example:

using System.ComponentModel.DataAnnotations;

public class Employee

```
{  
    [Required]  
    public string Name {get; set;}  
    [Range (18, 60)]  
    public int Age {get; set;}  
    [EmailAddress]  
    public string Email {get; set;}  
}
```

CRUD WITH EF CORE: CRUD means:

C → Create, R → Read U → Update D → Delete

Create

```
dbContext.Employees. Add(employee);
```

```
dbContext.SaveChanges();
```

Read

```
var employees = dbContext.Employees. ToList ();
```

Update

```
employee. Name = "Devika";
```

```
dbContext.SaveChanges();
```

Delete

```
dbContext.Employees. Remove(employee);
```

```
dbContext.SaveChanges();
```

ROUTING: Routing determines Which controller/action should handle a particular URL.

Example:

/api/employees

could be mapped to:

EmployeesController

HTTP Route Examples

[HttpGet]

```
public IActionResult GetAll ()
```

```
{
```

```
    return Ok ();
```

```
}
```

URL:

GET /api/employees

For ID:

[HttpGet("{id}")]

```
public IActionResult GetById (int id)
```

```
{
```

```
    return Ok(id);
```

```
}
```

URL:

GET /api/employees/5

POST:

[HttpPost]

public IActionResult Create (Employee employee)

```
{  
    return Ok ();
```

```
}
```

URL:

POST /api/employees

Delete:

[HttpDelete("{id}")]

public IActionResult Delete (int id)

```
{  
    return Ok ();
```

```
}
```

URL:

DELETE /api/employees/5